

Project: SecondSelf — Your Personal AI Second Brain

ProblemStatement.md

Every notes app fails the same way: you capture hundreds of notes, bookmarks, PDFs, and ideas — and then you never find them again. Information goes in, but nothing comes back out. Notes sit in folders nobody re-reads. Bookmarks pile up unread. Knowledge doesn't compound.

Goal: Build an end-to-end system where you can capture *anything* (a note, a link, a file), have AI automatically classify and file it, auto-link it to related knowledge, render it as a live interactive graph you can explore, and — most importantly — *ask it any question in plain English and get an answer synthesized from your own accumulated knowledge*. Then deploy it to a public URL anyone can open.

Not a notes app. Not a chatbot. A brain that organizes itself and answers for you.

Final System (what you're building over 4 weeks)

Capture any note/link/file



AI classifies & files it (PARA method)



AI auto-links it to related notes (embeddings)



Everything renders as a live, interactive, hoverable graph



Ask it anything in plain English → answer pulled from YOUR notes



Deployed on a public URL anyone can open

Week-by-Week Problem Statements

Each week is a self-contained problem. Build it, test it on **real** data (your own notes — not test data), and each week's output becomes the next week's input.

Week 1 — The Archivist: "Capture Everything, Lose Nothing"

Problem

You have no single place to put things. Ideas, links, and notes scatter across apps, browser tabs, and your memory. Build the foundation: **one command that captures anything into one place.**

Build

1. Set up the project structure from scratch:
 - raw/ — where every raw capture lands
 - wiki/ — (used later) organized, linked notes
2. Write a Python capture script that takes any note, link, or file and saves it into raw/ with:
 - a **timestamp**
 - a **unique ID**
 - the raw content
3. Test it on **10+ real** pieces of your own scattered information.

Deliverable ("Ship the Capture Pipeline")

- A working capture script — one command saves anything to raw/ with timestamp + unique ID.
- Your raw/ folder populated with **10+ real captured items** (not test data).
- 🏆 **Badge: The Archivist**

Acceptance Criteria

- [] raw/ and wiki/ folder structure exists
- [] One command captures a note, a link, AND a file
- [] Every capture has a timestamp + unique ID
- [] 10+ real items captured

Week 2 — The Librarian: "Teach AI to Organize For You"

Problem

A pile of raw captures is still a mess. Manual tagging never happens. Make the AI do the filing — and make it notice when two notes are about the same thing and link them automatically.

Build

2.1 — Auto-Classify (The Sorting Hat)

- Write a function that sends any raw capture to a free LLM (Groq / Llama 3) and gets back:
 - a **category** (using the **PARA** framework: Projects, Areas, Resources, Archives)
 - **tags**
 - a **one-line summary**
- Run it across last week's real captures and watch them organize themselves.

2.2 — Auto-Link Related Notes (Connect the Dots)

- Compute **embeddings** for each note (sentence-transformers, local + free).
- Compare each new capture against existing notes in wiki/.
- When content is related (similarity above a threshold), **auto-insert a link** between them.
- No manual tagging — the system notices relationships on its own.

Deliverable ("Ship the Self-Organizing Wiki")

- A pipeline that auto-classifies raw captures with PARA and auto-links related notes.
- Run on **15+ real items** → an organized wiki/ folder with linked notes.
- 🏆 **Badge: The Librarian**

Acceptance Criteria

- [] Any raw capture → category + tags + summary automatically
- [] PARA categorization working
- [] Embeddings computed per note
- [] Related notes auto-linked (no manual tagging)
- [] Runs on 15+ real items → organized wiki/

Week 3 — The Cartographer: "Visualize the Brain"

Problem

Your knowledge is now organized and linked — but you can't see it. Turn the wiki into something you can actually look at, explore, and watch think.

Build

3.1 — Graph Data Model (Give It a Shape)

- Write a script that reads every note and its links.
- Build a **nodes-and-edges** representation in memory:
 - every note → a **node**
 - every relationship/link → an **edge**
- Export it as clean **JSON**.

3.2 — Interactive Graph (The Brain Comes Alive)

- Use a JS graph library (**vis-network** or **Cytoscape.js**) to render:
 - notes as nodes (that pulse / are visually alive)
 - links as edges
 - **hover popups** that reveal each note's content
 - **drag-to-explore** and **zoom**
- A **force-directed** graph of your own knowledge.

Deliverable ("Ship the Living Brain")

- Your wiki converted to a graph and rendered as an interactive visual brain (hover, drag, zoom), built from your real notes.
- 🏆 **Badge: The Cartographer**

Acceptance Criteria

- [] Script builds nodes + edges from notes and exports clean JSON
- [] Interactive force-directed graph renders from that JSON
- [] Hover reveals note content
- [] Drag + zoom work
- [] Built from your real notes, not dummy data

Week 4 — The Oracle: "Ask It Anything, Ship It Public"

Problem

A visual brain is beautiful, but the real payoff is **answers**. Wire up natural-language search over everything you know — then package the whole thing into one deployable product.

Build

4.1 — Ask Your Brain (Natural Language Search)

- Build a single ask() function that combines:
 - the **embeddings** (find relevant notes to a question)
 - the **wiki** (the source content)
 - an **LLM** (synthesize an answer from retrieved notes)
- This is retrieval-augmented Q&A over your own knowledge.
- Test against real questions about your own captured notes.

4.2 — UI, Deployment, Public URL (Give It a Face)

- Assemble everything into **one Streamlit app**:
 - the interactive brain graph
 - the ask-anything search bar
- Deploy to a free platform (**Streamlit Cloud / HF Spaces**).
- Get a **public URL** anyone can open.

Deliverable ("Ship SecondSelf" — the final product)

Deploy the complete system — capture → auto-classify → auto-link → live interactive graph → ask-anything search — all wired into one Streamlit app with a public URL.

- 🏆 **Badge: The Oracle**

Acceptance Criteria

- [] ask() returns answers synthesized from your own notes (retrieval + LLM)
- [] One Streamlit app contains both the graph and the search bar
- [] Deployed live with a public URL
- [] Full pipeline works end to end in the deployed app

Final Deliverables (whole project)

- [] Public **GitHub repo** with a clean README + setup instructions
- [] **Live deployed URL** — interactive graph + ask-your-brain search, both working
- [] End-to-end flow verified: capture → classify → link → graph → ask
- [] All 4 weekly milestones complete (Capture Pipeline, Self-Organizing Wiki, Living Brain, SecondSelf deployment)

Suggested Repo Structure

```
secondself/
├── raw/           # Week 1: raw captures (timestamp + unique ID)
├── wiki/          # Week 2: classified + auto-linked notes
├── capture.py     # Week 1: one-command capture
├── classify.py     # Week 2: PARA classification via LLM
├── link.py        # Week 2: embeddings + auto-linking
├── build_graph.py # Week 3: nodes/edges → graph.json
├── graph.json     # Week 3: exported graph data
├── ask.py         # Week 4: retrieval + LLM answer
├── app.py         # Week 4: Streamlit UI (graph + search)
├── requirements.txt
└── README.md
```

Suggested Build Order in Cursor

1. Scaffold repo structure + requirements.txt
2. capture.py → test on real items (Week 1)
3. classify.py → PARA categories/tags/summary (Week 2.1)
4. link.py → embeddings + similarity auto-linking (Week 2.2)
5. build_graph.py → JSON nodes/edges (Week 3.1)
6. Graph render with vis-network/Cytoscape (Week 3.2)
7. ask.py → retrieval-augmented Q&A (Week 4.1)
8. app.py → Streamlit app combining graph + search (Week 4.2)
9. Deploy to Streamlit Cloud / HF Spaces → public URL
10. Write README, push to GitHub

AI Tools

- <https://cursor.com/download>
- <https://antigravity.google/product/antigravity-ide>
- <https://qoder.com/>
- <https://devin.ai/download/>
- VS code + Claude

Steps

Context

- problemStatement.txt → store the problem that you are solving
- architecture.md → HOW we will build the project
- implementation-plan.md → phase-wise implementation plan
 - Phase 0 → setup
 - Phase 1-5 → Implement code of the project
 - Phase 6-7 → locally test
 - Phase 8-9 → deploy the project + final round of testing
- edge-case.md → corner scenarios and edge-case

Prompts

1. Generate a detailed architecture for making this project using @PROBLEM_STATEMENT.md
2. Save the architecture into architecture.md file
3. Generate a phase-wise Implementation-plan.md using @architecture.md and @PROBLEM_STATEMENT.md
4. Generate an edge-case.md for storing all corner scenario and edge-cases for this project using @docs/architecture.md and @docs/Implementation-plan.md
5. Implement phase0 as per the @docs/Implementation-plan.md